The covtl-pipeline Dynamic Polar Visualization

Two-branch display of the COVTL articulatory trajectories
(vocalic background and consonantal excursions)

Frédéric Berthommier

Version 1.1.0 — September 2026

Scope. This manual documents the *dynamic polar figure* of the covtl-pipeline articulatory synthesizer (module `vtl_synth/video/polar_video.py`): the polar coordinate model behind it, the two-branch reconstruction of the vocalic and consonantal trajectories from the engine's own complex values, the `.polar` intermediate file, and the dual-panel MP4 output. It is one of the five manuals of the repository, a companion to the main manual (`docs/manual.pdf`), the speaker-registry manual (`docs/speakers.pdf`), the language-pack manual (`docs/language_pack.pdf`) and the expressive-prosody guide (`docs/expressive_prosody.pdf`).

Contents

1 Introduction **1**

1.1 What the polar figure shows . . . 1

1.2 Why two branches . . . 2

2 The polar coordinate model **2**

2.1 Projection and targets . . . 2

2.2 Arcs in the complex plane . . . 3

2.3 Superposition . . . 3

3 From engine to figure: the two-branch reconstruction **3**

3.1 Instrumented re-run of the engine . . . 3

3.2 Branch reconstruction rules . . . 4

3.3 The .polar intermediate file (version 4) . . . 4

4 Rendering the figure **5**

5 Using the visualization **5**

5.1 Command line . . . 5

5.2 Python API . . . 6

6 Worked example and validation **6**

7 Files, dependencies and demos **8**
7.1 Module and dependencies 8
7.2 Files 8
7.3 Regenerating the demos 8
7.4 Caveats 8

1 Introduction

1.1 What the polar figure shows

The covtl-pipeline synthesizes speech by driving VocalTractLab with articulatory trajectories computed in *polar coordinates* (ρ, θ): every phoneme of the inventory carries a target point — a radius ρ (cm) and an angle θ (rad) in the midsagittal plane — and the trajectory engine moves an abstract articulator through these targets along curved arcs. The polar figure makes this hidden machinery visible: it replays, as an animated curve, the path the engine actually takes.

Because the model is a *superposition* model, a single path is not enough. At every instant two gestures run simultaneously:

- the **vocalic branch** $z_{\mathrm{v}}(t)$ (drawn in **red**), the vowel-to-vowel background that carries the syllable nucleus, moving from anchor to anchor;
- the **consonantal branch** $z_{\mathrm{c}}(t)$ (drawn in **blue**), which exists only inside consonant clusters: it *detaches* from the vocalic background, visits the consonant targets $C_1, \ldots, C_m$ one after another, and returns to the vocalic path.

Figure 1 shows the two branches over the whole utterance `ba.da.ga`: the red background sweeps between the vowel anchors while the blue excursions leave it for /b/, /d/ and /g/ and come back. During each cluster the two curves are *simultaneous and dissociated* — the defining property of the display.

1.2 Why two branches

The displayed figure mirrors the synthesis architecture. Reconstructing a single path by *inverting* the 15 parameter frames actually sent to the synthesizer (a least-squares fit of the polar projection) cannot represent the superposition: inside a cluster the parameter frames are a blend of two gestures and lie *off* the polar manifold, so the fit zigzags and the two branches collapse into one visible path. The module therefore rebuilds both branches *directly from the engine's own polar points*, using the exact arc machinery of the reference model (Section 3). The inversion function `polar_from_pval` is still shipped, as a diagnostic tool only.

2 The polar coordinate model

2.1 Projection and targets

The polar projection maps a target (ρ, θ) onto the synthesizer parameter space. For every parameter p of the 15 COVTL parameters, with calibration constants $c_0^{(p)}, c_1^{(p)}, c_2^{(p)}$ (columns of the matrix `CO`):

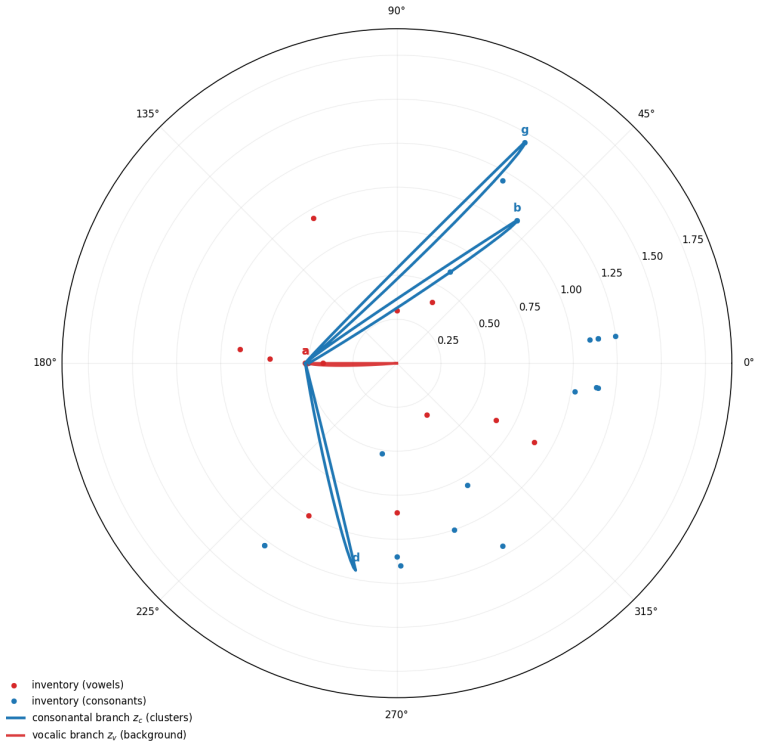


Figure 1: Static view of the two branches for `ba.da.ga` (JD3 speaker, internal notation). Red: the vocalic background $z_{\mathrm{v}}(t)$, always active. Blue: the consonantal branch $z_{\mathrm{c}}(t)$, active only during the three clusters; each blue excursion peaks exactly on the consonant target (large dots: phoneme inventory; small labels: phoneme keys at their targets).

$$P_p(\rho, \theta) \;=\; c_1^{(p)} \;+\; \rho\, c_0^{(p)} \cos\!\big(c_2^{(p)} - \theta\big). \tag{1}$$

Vowel targets come from the active speaker calibration (`VOWEL_TARGETS` in the installed constants, e.g. /a/ at $\rho = 1.00$, $\theta = \pi$ on jd3/en); consonant targets from the consonant table plus contextual place rules (dental /d/, velar /g/ resolved by vowel context, cluster-specific adjustments). In the complex plane a target is the complex number $z = \rho\, e^{i\theta}$.

2.2 Arcs in the complex plane

All motion is produced by one primitive, `polar.polar_arc`, which interpolates between a departure point $z_1 = \rho_1 e^{i\theta_1}$ and an arrival point $z_2 = \rho_2 e^{i\theta_2}$. Writing th for a half-turn sweep from 0 to π (or π to 0 when `orientation='inverse'`, which is the departure-to-arrival direction used by the engine and the display) and $r_k = \cos^2(th/2)$:

$$z(th) \;=\; (1 - r_k)\, z_1 \;+\; r_k\, \rho_2\, e^{\,i\left(\theta_2 + \frac{\nu}{K}\, th\right)}. \tag{2}$$

Three parameters control the shape of the path:

- **Curvature K.** The arrival angle carries an extra phase ν/K; the smaller K, the more the arc bows away from the straight chord. The display uses $K = K_{\mathrm{VOWEL_DEFAULT}} = 30$ (Berthommier v15g reference), which gives a visible but moderate curvature (sagitta 2–2.4 % of the chord on the `ba.da.ga` validation corpus). Note that the trajectory *engine* drives its

parameter arcs with $K = 1000$ (quasi-rectilinear); using $K = 30$ on screen is a deliberate visualization choice that restores the geometry of the reference model.

- **Direction ν.** $\nu = +1$ bows the vocalic arcs one way, $\nu = -1$ the consonantal sub-arcs the other way, as in the reference CV superposition figure.
- **Angular path.** The arrival angle is unwrapped so the arc always takes the shortest angular route (no $-\pi \leftrightarrow +\pi$ wrap-around jumps).

Stationary holds (vowel plateaus, decay/attack anchors) are constant complex samples produced by `polar.stationary_point`. Both primitives sample at 100 Hz, the gesture rate of the engine (1 step = 10 ms).

2.3 Superposition

At every instant the final parameter frame sent to the synthesizer is a blend: the *selector* parameters owned by the active consonants follow the consonantal branch, while all other parameters follow the vocalic background (function `build_cluster_pval` of `syltraj.py`). The figure mirrors this architecture by keeping the two branches separate on screen instead of merging them into one path.

3 From engine to figure: the two-branch reconstruction

3.1 Instrumented re-run of the engine

The figure must show *exactly* the timing of the synthesis. The pipeline therefore re-runs the same engine function (`syltraj.build_phrase_pval`, with the same normalization and the same duration parameters) while temporarily wrapping the five helpers of `build_global_pval` that carry the polar geometry. Each helper records, per appended block and in `block_info` order, the polar points the engine drives toward:

Recorded event	Content
`arc`	departure and arrival anchor pair + duration in steps (pause, initial and terminal arcs)
`bg`	anchor pair + duration (vocalic background arcs; the schwa routing of closing VV arcs records its two half-arcs)
`hold`	anchor point + duration (decay/attack stationary holds)
`cluster`	anchor pair, list of consonant *node* targets $[(\rho_k, \theta_k)]$ (context-adjusted), sub-arc duration T

Vowel plateau blocks are appended inline by the engine (no helper to wrap); the reconstruction rebuilds them from `VOWEL_TARGETS[vowel_key]`. After the run, every queue must be exactly consumed by a pass over `block_info`; any leftover event or duration mismatch raises a `RuntimeError` (alignment drift guard), so a silent desynchronization of the figure with respect to the audio is impossible by construction.

3.2 Branch reconstruction rules

The pass over `block_info` emits, at 100 Hz, the two aligned trajectories:

Block kind	**Branch behavior**
plateau	red: stationary vowel target; blue: —
background, pause, initial, terminal	red: arc(dep, arr) with K=30, ν=+1; blue: —
decay, attack	red: stationary anchor; blue: —
cluster, $n = (m{+}1)\,T$ steps	red: arc(dep, arr) over the whole block (the background keeps flowing anchor to anchor); blue: sub-arcs $dep \to C_1 \to \cdots \to C_m \to arr$, T steps each, drawn in the *original arcplot anchoring* ($z(\theta) = \rho(\theta)\,p_a\,e^{i\nu\theta/K} + (1{-}\rho(\theta))\,p_d$, $\rho = \cos(\theta/2)$, with the phase carrier p_a = the *shorter-radius* endpoint of each leg, K=10, ν=−1) — a closed V→C→V pair renders as the teardrop of the article's planning figures, not as a consonant-side lens

Two properties deserve emphasis. First, *during a cluster the red branch is not interrupted*: the vocalic background runs underneath the consonant excursion (this is the `bg` component of `build_cluster_pval`). Second, the blue branch peaks *exactly* on the consonant node targets at the end of each approach sub-arc — on the validation corpus `ba.da.ga` the reconstructed peaks coincide with the engine targets to within 10^{-9} (Table 1).

3.3 The .polar intermediate file (version 4)

The result is stored in a self-documented JSON file with `format` = `covtl-polar` and `version` = 4:

```
{
  "format": "covtl-polar", "version": 4,
  "sample_rate": 100, "speaker": "JD3",
  "n_steps": 176, "duration_s": 1.76,
  "columns": { "vocalic_rho": "...", "consonantal_theta": "...", ... },
  "orientation": "trigonometric: theta=0 East, counter-clockwise",
  "trajectory": {
    "vocalic":     { "rho": [...], "theta": [...] },
    "consonantal": { "rho": [null, ..., 0.96, ...],   // null = inactive
                     "theta": [null, ..., 0.87, ...] }
  },
  "phonemes":  [ { "key": "b", "category": "C", "rho": 1.06,
                   "theta": 0.87, "t0": 20, "t1": 36 }, ... ],
  "inventory": [ { "key": "a", "category": "V", ... }, ... ]
}
```

The two 100 Hz trajectories are the single source of the figure — the video never re-derives them from the `.tract`. The consonantal arrays carry `null` while the branch is inactive (outside clusters), which lets the renderer break the blue curve cleanly. Phoneme timing is in gesture steps (10 ms per step); `t0`/`t1` are exact multiples of the block boundaries, so the `.polar`, `.tract` and `.wav` timelines agree by construction ($n_{\text{tract frames @400 Hz}} = 4 \times n_{\text{steps}}$).

4 Rendering the figure

The final MP4 is a dual-panel composition at 25 Hz (H.264 + AAC, audio muxed from the synthesized `.wav`):

- **Left panel**: the standard sagittal tract view, rendered from the `.tract` file (the standard video of the pipeline).

- **Right panel**: the polar figure, in trigonometric orientation ($\theta = 0$ East, counter-clockwise), with: the two branch curves; a fading trail of ≈ 0.75 s per branch (red and blue independently); a current-position dot per active branch; the phoneme inventory as small permanent dots (red vowels, blue consonants); a thin segment from the current position toward the active target; a transient label of the current phoneme (internal input notation, e.g. `tS`, `a~`) displayed 0.5 s from the onset of the phoneme near its target.

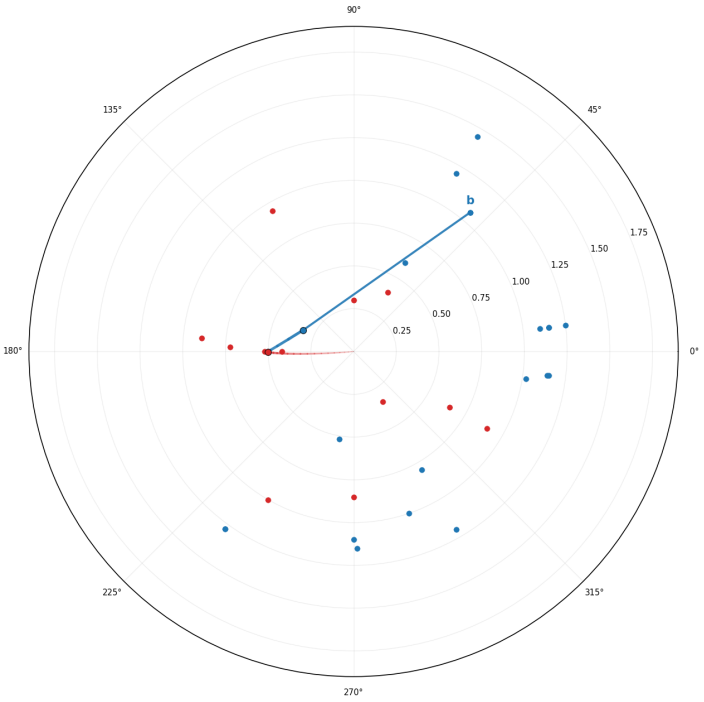


Figure 2: Rendered frame at $t = 0.24$ s in `ba.da.ga`, during the /b/ cluster: the blue branch has detached from the red background and reaches the /b/ target (upper right, $\rho = 1.06$, $\theta \approx 0.87$ rad), while the red background continues its progression toward the vowel anchor.

5 Using the visualization

5.1 Command line

The feature is carried by three subcommands of `vtl-synth`; the interfaces are identical to the rest of the pipeline:

```
# Full chain: .tract + .wav + .polar + dual-panel .mp4
vtl-synth run --polar "this is easy for us" -l en -o out --label polar_demo_en

# .polar only, from a phrase in the internal notation
vtl-synth polar-build "ba.da.ga" -o out/ba.polar --tcons 160

# Dual-panel video from existing files
vtl-synth polar-video out/polar_demo_en.tract --polar out/polar_demo_en.polar
    \
        out/polar_demo_en.wav -o out/polar_demo_en.mp4
```

Reproducing the reference demos requires the **JD3 speaker** to be the active speaker (`python install_speaker.py install jd3`; restore your previous speaker afterwards). The engine

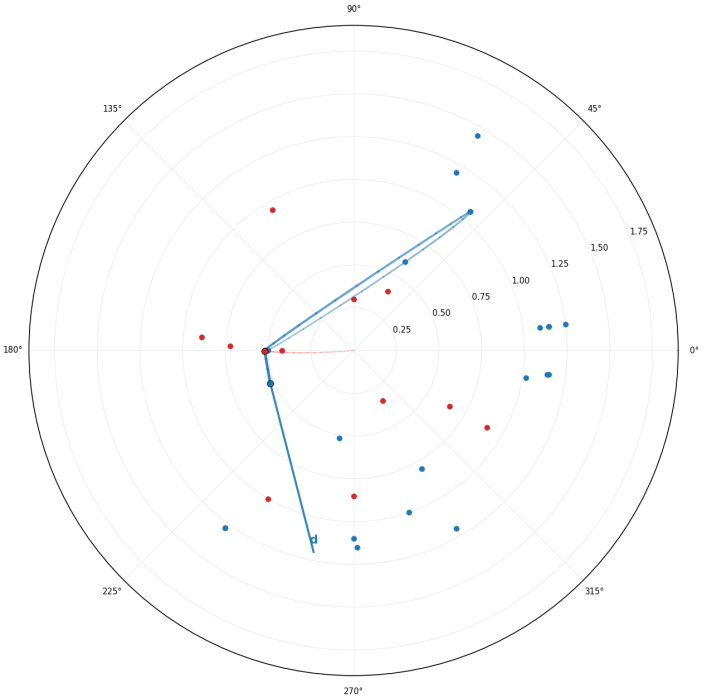


Figure 3: Rendered frame at $t = 0.64$ s, during the /d/ cluster: the fading blue trail of the /b/ excursion is still visible (memory ≈ 0.75 s) while a new blue excursion approaches the /d/ target (bottom, $\rho = 1.2$, $\theta \approx -1.77$ rad); the red background stays on the vowel axis. The two branches are simultaneous and dissociated.

options (`-l`, `-tcons`, `-tvoy`, `-tpause`, `-f0`, `-s`) behave exactly as in `run` without `-polar`, because the `.polar` build re-runs the same engine with the same parameters.

5.2 Python API

```
from vtl_synth.video.polar_video import (
    build_polar, write_polar, read_polar,
    render_polar_frame, make_polar_video)

# 1. Build the two-branch trajectories (timings of the synthesis)
pol = build_polar(["ba.da.ga"], t_cons=16, t_voy=8,
                  pause_short_ms=200.0, pause_long_ms=320.0)
write_polar("out/ba.polar", pol)

# 2. Read it back
pol = read_polar("out/ba.polar")
zv, zc = pol["traj_v"], pol["traj_c"]   # (n, 2) rho/theta, NaN = inactive

# 3. Draw one frame (matplotlib polar axes) or the whole MP4
import matplotlib.pyplot as plt
fig = plt.figure(); ax = fig.add_subplot(111, projection="polar")
render_polar_frame(ax, pol, t=0.64)
make_polar_video("out/ba.tract", "out/ba.polar", wav_path="out/ba.wav",
                 out_path="out/ba.mp4")
```

6 Worked example and validation

The validation corpus is `ba.da.ga` with $T_{\text{cons}} = 16$ steps (160 ms), $T_{\text{voy}} = 8$ steps, pauses 200/320 ms, JD3 speaker. Table 1 compares the reconstructed blue-branch peaks with the engine consonant targets; Figure 1 (page 2) shows the full utterance.

Table 1: Blue-branch peaks vs. engine consonant targets on `ba.da.ga` (JD3).

Consonant	Engine target (ρ, θ)	Reconstructed peak	Error	Sagitta (% chord)
/b/	(1.06, 0.87)	(1.060, +0.873)	$< 10^{-9}$	2.0
/d/	(1.20, −1.77)	(1.200, −1.767)	$< 10^{-9}$	2.4
/g/	(1.45, +1.05)	(1.450, +1.047)	$< 10^{-9}$	2.2

Automated checks:

- **Targets.** every blue sub-arc terminates exactly on its node target (angle compared modulo 2π).
- **Simultaneity.** the blue branch is active on every step of each cluster block ($(m{+}1)\,T$ steps: approach *and* release), and only there.
- **Background continuity.** the red branch is finite on every step of the utterance and rests exactly on the vowel target during plateaus.
- **Curvature.** the sagitta of every red arc is between 2 % and 2.4 % of its chord ($K = 30$; the engine's $K = 1000$ would give $\approx 0.07\,\%$, invisible). The blue sub-arcs (arcplot anchoring, $K = 10$, phase on the shorter-radius endpoint) show the teardrop profile: belly at ≈ 0.35 of the chord from the shorter-radius endpoint, tapering into the node target (measured 0.34–0.36 on `bi/ba`, matching the reference display implementation of arXiv:2307.02299's figures).
- **Timeline.** `.polar` duration = `.tract` duration = MP4 duration (within one 25 Hz frame); the test selection `pytest tests -k "video or pipeline or cli or polar"` passes unchanged (9 tests).

7 Files, dependencies and demos

7.1 Module and dependencies

The feature is additive and self-contained: the synthesis path (`core/polar.py`, `core/syltraj.py`) is only *read* (temporary instrumentation, restored in a `finally` block), so the engine output and the regression baselines are untouched. `Pipeline.run(...)` keeps its signature; the figure is activated by the keyword `polar=True`, and the three CLI forms (`run -polar`, `polar-build`, `polar-video`) reuse the engine options (`-l`, `-tcons`, `-tvoy`, `-tpause`, `-f0`, `-s`) unchanged. The only dependency is `matplotlib`, required by `run -polar` (already listed for `plot-tract`):

```
[project.optional-dependencies]
plot  = ["matplotlib"]
video = ["Pillow", "imageio-ffmpeg", "matplotlib"]
```

7.2 Files

Path	Content
`vtl_synth/video/polar_video.py`	the polar-figure module (build, read/write, frame rendering, dual-panel composition)
`docs/polar_visualization.tex/.pdf`	this manual
`docs/figures/fig_.png`	the figures of this manual
`examples/`, `out/`	showcase demos, regenerable (below)

The demo *media* (`.mp4` ≈ 0.3–5 MB each, plus `.tract`/`.wav`/`.polar`) may be committed as small showcase files or regenerated on demand; prefer **Git LFS** for the MP4s (`git lfs track "*.mp4"`).

7.3 Regenerating the demos

A minimal script regenerates the two showcase videos (JD3 must be the active speaker, `python install_speaker.py install jd3`; restore the previous speaker afterwards):

```
from vtl_synth import Pipeline
for label, kwargs in [
    ("polar_demo_en", dict(text="this is easy for us", use_g2p=True)),
    ("polar_e2e", dict(text="ba.da.ga", use_g2p=False)),
]:
    Pipeline(f0_hz=102.216, speaker="JD3", t_cons_ms=100.0,
             t_voy_ms=80.0).run(
        kwargs["text"], output_dir="out", use_g2p=kwargs["use_g2p"],
        label=label, polar=True)
```

7.4 Caveats

- **Speaker dependence.** the figure uses the polar targets of the *active* speaker constants. The reference demos are JD3; other speakers (s1, s2, m01, w02) work but their targets differ (e.g. /b/ is $(0.96, 4.90)$ for s2); see `docs/speakers.pdf`.
- **Speaker switching.** generating the demos requires installing JD3 (`install_speaker.py`), which swaps the shared wheel resource; restore the previous speaker afterwards.
- **Display curvature.** $K = 30$ is a visualization constant (reference-model geometry); the parameter arcs of the engine remain $K = 1000$. This is intentional and documented in the module header.
- **Work-directory hygiene.** the dual-panel encoder purges stale `frame_*.png` files in its temporary work directories before rendering and verifies the composed frame count before encoding, so a video can never outlive its audio because of leftover frames of an earlier render.

Reference model: F. Berthommier, "COVTL" technical reports v15g, arXiv:2307.02299 (2023) and JEP 2024, pp. 541–550.